FEAT-ZD-48600 · DE-03 Database changes · Completed database changes

Data Engineering agent · item task.db_changes · source hobasa-data-engineering-agent/outputs/FEAT-ZD-48600/task.db_changes.md · approved by Data engineer sim.data_engineer

task.db_changes: FEAT-ZD-48600

Database changes check for FEAT-ZD-48600 (ZD-48600, "Show which overdue invoices belong to vendors that usually pay late"), Data Engineering lane, repository ap_pipeline. This is DE-03 Database changes (attempt 1).

Field	Value	Source
Trace ID	FEAT-ZD-48600	record:brief
Step	DE-03 Database changes (attempt 1)	record:brief
Workbook action	Check the required database changes are completed	record:brief
Owner / gate	Data engineer / automated checks, independent judge, sampled human evaluation	record:brief
Inputs	feat.tasks (b0fd4d2), feat.design (722a98f)	record:brief
Lane tasks in scope	TSK-01 (STORY-01, `paid_date` in `to_model`), TSK-02 (STORY-02, `delays_by_vendor`)	feat.tasks (section 1)

1. Result

No database changes are required for FEAT-ZD-48600, so none were made, and there is nothing outstanding for this step.

Check	Finding	Source
Does the design add storage, a database or a schema migration?	No. Storage, database and schema migration are listed as "Out of this design"	feat.design (A1, Boundary "Data")
Does the design allow new storage?	No. DD-1: "No new repository, service, runtime, library or storage"	feat.design (A7, DD-1)
Would storage reopen the platform design?	Yes. "any storage or network call from the libraries" re-triggers DES-01; the design triggers none	feat.design (A8)
Are the new functions in-memory?	Yes. NF-10: "Every new function is pure and in-memory: no I/O, no network, no mutation of inputs"	feat.design (A5, NF-10)
Where does the new `paid_date` data live?	As one new key on the in-memory Invoice record dict (DC-1) and one new reject reason `'paid_date'`; the delay history is an in-memory `Dict[str, List[int]]` (DC-2)	feat.design (A1, A3 DC-1 and DC-2, B1.3)
Do the DE tasks touch any persistence file?	No. TSK-01 budget: `transform.py`, `__init__.py` (version only), `tests/test_pipeline.py`. TSK-02 budget: new `delays.py`, new `tests/test_delays.py`	feat.tasks (TSK-01, TSK-02 "Files in budget")
Does ap_pipeline have a database today?	No. "ap_pipeline has no database; no migrations exist or are required"	tool:repo-access
Code read confirms	`pipeline.run` returns `(records, report)` in memory from `parse_feed` → `to_model` → `quality_report`; no database call	file:ap_pipeline/src/ap_pipeline/pipeline.py

Check	Finding	Source
Code read confirms	`to_model` builds plain dicts with `id`, `vendor`, `due_date`, `amount`, `paid` and returns `(records, rejected)`; no database call	file:ap_pipeline/src/ap_pipeline/transform.py
Repository search	No match in the ap_pipeline checkout for database libraries or DDL (`sqlite`, `sqlalchemy`, `psycopg`, `create table`, `alter table`, `migration`, `databricks`, `cursor`, `connect()`. The repository has no migrations folder	file:ap_pipeline (Grep and Glob over the checkout)

2. Database change list

Change	Object	Status
None required	None	Not applicable

The `paid_date` field is a data-model addition (Invoice record key), not a database column. Recording it in the Data Model belongs to DE-05 Data Model Changes (feat.data_model_change), not to this step.

3. Notes for later steps

- DE-05 should record the `paid_date` key (`date` or `None`) on the Invoice record and the new reject reason `'paid_date'` (feat.design B1.3).
- `schedule.json` is unchanged (feat.design B1.1), so this feature needs no change to the schedule.
- If a later decision adds storage, or a transport between ap_aging and ap_portal (DQ-1, TQ-3), DES-01 reopens and this step re-runs on that trigger (feat.design A8).

4. Open questions

None. No input is missing or contradictory for this step.

Tool results (recorded by the spine from the real tools)

[Tool result] [ap_pipeline] No database in this repository's architecture; no migrations required